

OPTIMA

Intelligent Test-Case Optimization & Coverage Engine

HACKATHON PRESENTATION

CORE	Sorting • Greedy • 0/1 Knapsack
STACK	React • Python Flask • SQLite
DOMAIN	Software Testing & Algorithmic Optimization

Team Members: _____

Project Introduction

OPTIMA

Intelligent Test-Case Optimization & Coverage Engine

One-Line Idea: Select the most valuable software test cases under a limited execution-time budget while maximizing topic and edge-case coverage.

Problem Domain	Software Testing • Algorithmic Optimization • Test Automation
Core Algorithms	Sorting • Greedy • 0/1 Knapsack
Technology	React • Python Flask • SQLite
Team Members	_____

The Real-World Problem

- Modern applications can contain hundreds or thousands of test cases.
- Testing teams often have limited execution time before a release.
- Running the entire suite can be expensive and inefficient.
- Different test cases have different priorities and execution costs.
- Critical edge cases should not be missed.

Core Question

If execution time is limited, which test cases should be run to achieve maximum useful coverage?

Why It Matters

Example: Online Food-Delivery Application

- Login validation
- Payment failure
- Invalid card
- Empty address
- Server timeout
- Coupon validation
- Order cancellation
- Session expiry

Scenario: Complete suite = 2 hours | Available time = 10 minutes

OPTIMA identifies a compact set of high-value tests that fits the available execution budget.

Key Challenges

01	Limited Execution Budget — Only a fixed amount of time is available.
02	Large Test Suite — Hundreds or thousands of candidates may exist.
03	Different Priorities — Not every test case has equal importance.
04	Edge Cases — Rare but critical scenarios must be considered.
05	Optimal Selection — High priority alone may not produce the best combination.

What Makes OPTIMA Different?

OPTIMA combines multiple DAA techniques instead of relying on a single ranking rule.

**TEST CASES → RANKING → OPTIMIZATION → GREEDY vs KNAPSACK → COVERAGE
→ OPTIMIZED TEST SUITE**

Unique Approach: Compare a fast Greedy strategy with a 0/1 Knapsack optimization strategy under the same execution budget.

Key Features

✓ Test Case Repository	✓ Add / Delete Test Cases
✓ SQLite Persistence	✓ Priority & Time Based Sorting
✓ Greedy Optimization	✓ 0/1 Knapsack Optimization
✓ Greedy vs Knapsack Comparison	✓ Topic Coverage & Edge-Case Coverage
✓ Algorithm Decision Trace	✓ Performance Analytics
✓ PDF & Excel-Compatible Reports	✓

System Workflow

01 TEST CASE INPUT



02 SORTING



03 GREEDY + KNAPSACK



04 COMPARISON



05 COVERAGE ANALYSIS



06 FINAL TEST SUITE



07 ANALYTICS & REPORT

User Journey

Step 1	Add test cases with name, topic, edge-case status, execution time and priority.
Step 2	Set the execution budget.
Step 3	Run Sorting, Greedy and Knapsack.
Step 4	Compare algorithm results.
Step 5	Measure topic and edge-case coverage.
Step 6	View analytics and export the result.

Technology Stack

Frontend	React + Vite Interface, controls and analytics
UI	Custom CSS + Framer Motion Interactive visual experience
Backend	Python Flask REST API and processing
Database	SQLite Persistent test-case storage

Main Algorithms

SORTING	Ranks and organizes test cases by priority, execution time or efficiency.
GREEDY	Makes the best locally feasible selection at each step for fast optimization.
0/1 KNAPSACK	Treats each test case as selected/not selected and seeks maximum value without exceeding the budget.

Complexity

Algorithm	Purpose	Complexity
Sorting	Rank test cases	$O(n \log n)$
Greedy	Fast selection	$O(n \log n)$ with sorting
0/1 Knapsack	Budget optimization	$O(n \times B)$
Coverage Analysis	Measure coverage	$O(n)$

n = number of test cases; B = execution budget.

Live Demo

- Landing Page → Enter Engine
- Test Case Repository → Add test cases
- Sorting → Priority / Time / Efficiency
- Greedy → Set budget and run
- Knapsack → Run optimization
- Comparison → View both results
- Coverage → Topic and edge-case coverage
- Analytics → Charts and metrics
- Export → PDF / Excel-compatible report

Add screenshots of the actual OPTIMA interface on this slide.

Sample Result

Illustrative demo values — replace with actual final-run output if different.

Metric	Greedy	0/1 Knapsack
Selected Cases	4	5
Execution Time	9 min	10 min
Total Value	30	35
Coverage	75%	90%

Practical Impact

Efficient Testing	Focus execution on valuable test cases.
Better Coverage	Consider both topics and edge cases.
Algorithmic Decision Making	Replace arbitrary manual selection with measurable strategies.
Transparency	Show why cases were selected or rejected.
Measurable Results	Compare time, value and coverage.
Reporting	Export optimization results for review.

Current Achievement

- Working test-case repository
- Sorting engine
- Greedy optimizer
- 0/1 Knapsack optimizer
- Algorithm comparison
- Coverage analysis
- Analytics dashboard
- Report export

Future Improvements & Scalability

- Risk-based test prioritization
- Historical failure-rate analysis
- Automatic test-case generation
- CI/CD integration
- Jenkins / GitHub Actions integration
- Large-scale test-suite optimization
- Historical comparison of optimization runs

Future Vision: Evolve OPTIMA into an automated test-selection component for continuous software delivery pipelines.

Final Takeaway

OPTIMA transforms test-case selection from a manual decision into a measurable DAA-based optimization process.

OPTIMA

“Test smarter, not just more.”

THANK YOU • QUESTIONS & ANSWERS